Exp 6

Write a program to implement Error Detection And Correction using Hamming Code

Aim :-

To write a program to implement error detection and correction using the Hamming code.

Algorithm :-

1. Start the program
2. Read the number of 4 bit data blocks
   (n) Grouping the input data stream.
3. For each 4-bit data block D = d1 d2 d3 d4
   perform Hamming encoding
   a. Arrange 7 bits
   b. Compute parity bits
      P1 = d1 XOR d2 XOR d4
      P2 = d1 XOR d3 XOR d4
      P4 = d2 XOR d3 XOR d4
   c. Form 7 bit Hamming code word
      P1 P2 d1 P4 d2 d3 d4
4. Simulate transmission by introducing single bit error at chosen bit
5. At the receiver, recompute check bits
   Form received code word
   c1 = P1 XOR d1 XOR d2 XOR d4
   c2 = P2 XOR d1 XOR d3 XOR d4
   c4 = P4 XOR d2 XOR d3 XOR d4
6. Compute error position = c4 c2 c1

7. If error position = 0, no error has occurred. else, the bit at error position is inverted.

8. Extract original 4 data bits from the corrected 7 bit code word.

9. Repeat steps 3 to 8 for all 7 blocks

10. Stop the program.

Program :-

```
def encode(d):
    """ d = list of 4 bits [d1, d2, d3, d4] ->
    returns 7 bit hamming code list
Positions (1-indexed): 1=P1 2=P2 3=d1 4=P4
                       5=d2 6=d3 7=d4 """
    h = [0] * 7
    h[2] = d[0]
    h[4] = d[1]
    h[5] = d[2]
    h[6] = d[3]
    h[0] = h[2] ^ h[4] ^ h[6]
    h[1] = h[2] ^ h[5] ^ h[6]
    h[3] = h[4] ^ h[5] ^ h[6]
    return h

def detect_and_correct(h):
    c1 = h[0] ^ h[2] ^ h[4] ^ h[6]
    c2 = h[1] ^ h[2] ^ h[5] ^ h[6]
    c4 = h[3] ^ h[4] ^ h[5] ^ h[6]
    error_pos = c1*1 + c2*2 + c4*4
```

```
if error_pos != 0:
    print(f"Error detected: {error_pos} -> Correcting...")
    R[error_pos-1] ^= 1
else:
    print("No error detected")
return error_pos

def to_str(bits):
    return "".join(str(b) for b in bits)

def main():
    print("=Hamming code")
    n = int(input("Enter no. of 4 bit data"))
    for b in range(1, n+1):
        d = list(map(int, input("Enter 4 data bits (space separated):").split()))
        R = encode(d)
        print(f"Original Data bits = {to_str(d)}")
        print(f"Encoded code = {to_str(R)}")
    pos = int(input("enter bit position:"))
    if 1 <= pos <= 7:
        R[pos-1] ^= 1
        print(f"Transmitted code {pos}) = {to_str(R)}")
    else:
        print(f"Transmitted code (no error) {to_str(R)}")

    print(f"Received code : {to_str(R)}")
    print("Checking for errors")
    detect_and_correct(R)
print(f"Corrected code = {to_str(R)}")
```

Output :-

Enter number of 4 bit data Blocks in the stream : 2

Blok 1 Enter 4 data Bits : (0 1 1)

original data bits : 1011

Encoded Hamming code : 0110011

Enter bit position to introduce error (1-7, 0 = no error) : 3

Transmitted code : 0100011

Received code : 0100011

checking for errors...

Error detected at bit position : 3 → Correcting

Corrected code : 0110011

Recovered data Bits : 1011

Blok 2 Enter 4 data Bits : 0110

original data Bits : 0110

Encoded Hamming code : 1100110

Enter bit position to introduce error : 6

Transmitted Code : 1100100

Received code : 1100100

Checking for errors..

Error detected at bit position :

6 → correcting

Corrected code : 1100110

Recovered data bits : 0110

```
    rd = [R[2], R[4], R[5], R[6]]
    print(f"Recovered Data Bits: {to_str(rd)}")

if __name__ == "__main__":
    main()
```

Result :-

The program to implement error detection and correction using Hamming code concept was successfully written in Python, executed and verified with sample input data stream.